

# Assignment 0: Iterated Function Systems

作者：王洵

## 1 实验原理

### 1.1 迭代函数系统与吸引子

迭代函数系统 (Iterated Function System, IFS) 由有限个压缩仿射变换共同描述一类自相似分形。设变换集合为  $\{f_1, \dots, f_n\}$ , 其中每个  $f_i$  均为压缩映射 (contractive mapping), 则存在唯一吸引子 (attractor)  $A$  满足

$$A = \bigcup_{i=1}^n f_i(A).$$

式中  $f_i(A)$  表示集合  $A$  在变换  $f_i$  下的像。上式表明:  $A$  经整组变换作用后仍等于自身, 因而  $A$  是该系统的不动点。Barnsley's fern 给出直观例子——蕨叶的局部与整体在几何上相似, 这种自相似性可由若干旋转、平移、缩放等仿射变换编码。

### 1.2 仿射变换的矩阵表示

在二维情形下, 每个  $f_i$  可用齐次坐标下的  $3 \times 3$  矩阵表示。对点  $\mathbf{p} = (x, y)$ , 变换为

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & e \\ c & d & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix},$$

其中  $a, b, c, d$  构成线性部分 (旋转、缩放、错切等),  $e, f$  为平移分量,  $(x', y')$  为变换后的平面坐标。输入数据中给出上述矩阵, 以及每个变换被选中的概率  $p_i$ , 且满足  $\sum_{i=1}^n p_i = 1$ 。

### 1.3 随机迭代逼近

数值上对吸引子作随机迭代逼近。从单位正方形  $[0, 1]^2$  内随机取点  $\mathbf{x}_0$ , 再按概率  $\{p_i\}$  抽取变换并递推

$$\mathbf{x}_{k+1} = f_i(\mathbf{x}_k).$$

其中  $\mathbf{x}_k$  为第  $k$  次迭代后的点,  $f_i$  为本次抽中的变换。迭代足够多次后, 点落入吸引子附近。对大量独立起点重复该过程, 并将最终点画到图像上, 即可得到  $A$  的近似可视化。按概率加权抽样时, 对吸引子贡献更大的变换被选中的频率更高, 达到可接受图像质量所需的总点数随之减少。

## 2 程序设计与实现

### 2.1 总体架构

系统由线性代数库、图像读写库、IFS 模块与主程序组成：

- 线性代数库提供二维点与矩阵运算，用于表示采样点及仿射变换；
- 图像库负责初始化像素并写出输出图像；
- IFS 模块读入变换描述并执行随机迭代渲染；
- 主程序解析命令行参数，完成读入、渲染与保存。

主程序在解析参数并播种随机数后，依次调用读入、构造正方形图像、渲染与写出：

```
IFS ifs;
ifs.Input(input_file);

Image image(size, size);
ifs.Render(&image, num_points, num_iters);
image.SaveTGA(output_file);
```

### 2.2 IFS 模块的数据组织与读入

IFS 模块保存变换个数、仿射变换矩阵序列以及各变换的选择概率：

```
int n;
Matrix *transforms;
float *probabilities;
```

读入时先取得变换个数，再为矩阵与概率分配存储，并逐条读入概率  $p_i$  与对应的  $3 \times 3$  矩阵：

```
fscanf(input, "%d", &n);
transforms = new Matrix[n];
probabilities = new float[n];

for (int i = 0; i < n; i++) {
    fscanf(input, "%f", &probabilities[i]);
    transforms[i].Read3x3(input);
}
```

再次读入新描述前先释放旧数据；对象销毁时释放矩阵与概率占用的内存。

### 2.3 随机迭代渲染

渲染时先将整幅图像置为黑色背景，再对每一个样本执行下列步骤：

1. 在单位正方形  $[0, 1]^2$  内均匀随机采样起点  $\mathbf{v}$ ;
2. 迭代给定次数: 生成  $r \sim U(0, 1)$ , 按累积概率选出变换并对当前点施加仿射变换;
3. 将单位正方形坐标线性映射到像素坐标, 并在图像范围内写入白色点。

核心循环如下:

```
Vec2f v((float)rand() / RAND_MAX, (float)rand() / RAND_MAX);
for (int k = 0; k < num_iters; k++) {
    float r = (float)rand() / RAND_MAX;
    float sum = 0;
    int t = n - 1;
    for (int i = 0; i < n; i++) {
        sum += probabilities[i];
        if (r < sum) {
            t = i;
            break;
        }
    }
    transforms[t].Transform(v);
}
```

其中  $\text{sum}$  为截至当前变换的累积概率; 随机数  $r$  首次小于该累积和时选中对应变换, 再对点  $\mathbf{v}$  做齐次仿射变换。概率较大的变换被选中的次数更多, 吸引子上相应区域的采样更密。

二维点经矩阵升为齐次坐标  $(x, y, 1, 1)$ , 变换后再取回平面分量:

```
void Transform(Vec2f &v) const {
    Vec4f v2 = Vec4f(v.x(), v.y(), 1, 1);
    Transform(v2);
    v.Set(v2.x(), v2.y());
}
```

迭代结束后, 将点映射到像素:

$$x = \lfloor v_x \cdot (W - 1) \rfloor, \quad y = \lfloor v_y \cdot (H - 1) \rfloor,$$

其中  $W$ 、 $H$  分别为图像宽与高,  $v_x$ 、 $v_y$  为变换后点的横纵坐标。像素坐标落在图像范围内时写入白色。

## 2.4 主程序与运行参数

主程序识别下列命令行选项:

| 选项      | 含义         |
|---------|------------|
| -input  | IFS 描述文件路径 |
| -points | 随机起点（样本）总数 |
| -iters  | 每个样本的迭代次数  |
| -size   | 正方形输出图像的边长 |
| -output | 输出图像路径     |

解析完成后以当前时刻播种随机数生成器，再按 2.1 节流程完成渲染。

### 3 实验结果

#### 3.1 测试命令

Sierpinski triangle 场景在固定采样点数与分辨率下改变迭代次数：

```
# -input 输入 IFS 描述; -points 采样点数; -iters 每点迭代次数
# -size 图像边长; -output 输出图像路径
ifs -input <Sierpinski triangle 场景> -points 10000 -iters k -size 200 -output <对应输出>
```

Barnsley's fern 场景提高采样点数与分辨率：

```
ifs -input <Barnsley's fern 场景> -points 50000 -iters 30 -size 400 -output <对应输出>
```

#### 3.2 Sierpinski triangle：迭代次数对比

参数：采样点数 10000，图像边长 200，迭代次数  $k \in \{0, 1, 2, 3, 4, 30\}$ 。

图 3.2-1 ~ 3.2-3  $k = 0, 1, 2$

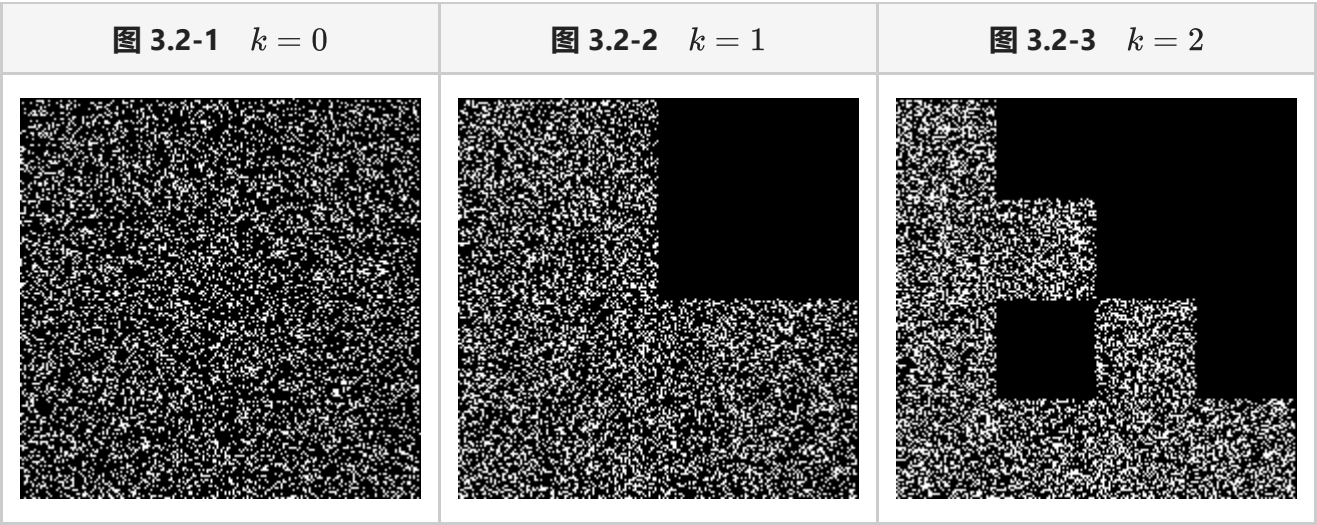
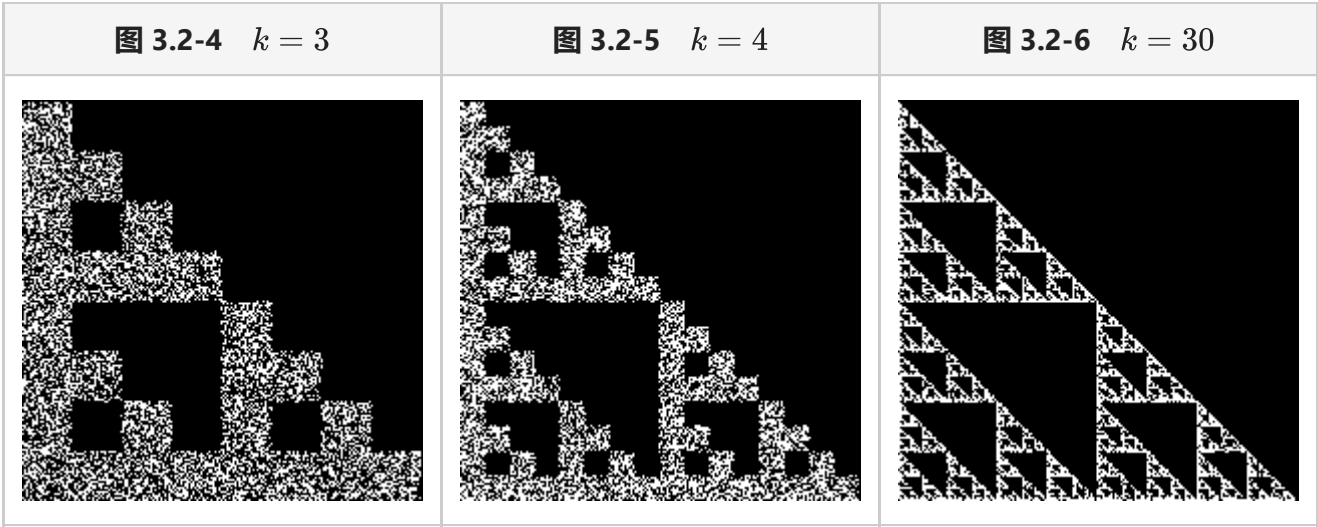


图 3.2-4~3.2-6  $k = 3, 4, 30$

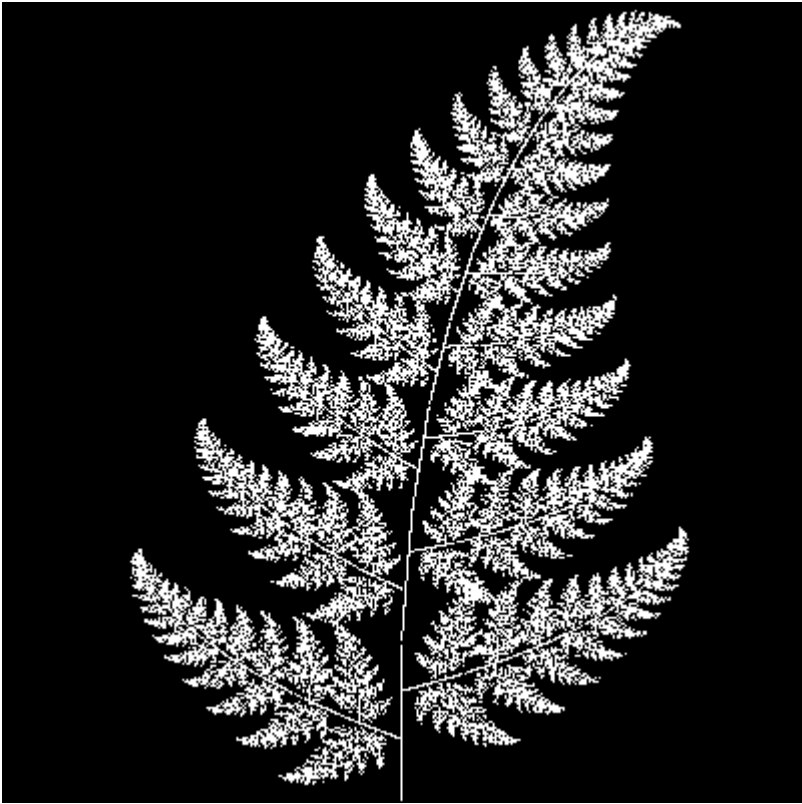


$k = 0$  时点仍为均匀随机起点，画面接近噪声。随着  $k$  增大，点逐步收缩到三个自相似子三角形； $k = 3、4$  时轮廓已可辨认， $k = 30$  时得到稳定的 Sierpinski triangle。该组结果说明：在变换与概率给定的前提下，迭代次数决定随机起点向吸引子收敛的程度。

3.3 Barnsley's fern

参数：采样点数 50000，迭代次数 30，图像边长 400。

图 3.3-1 Barnsley's fern 吸引子



fern

多组带概率权重的仿射变换在充分采样与迭代后收敛为蕨叶形态。与 3.2 节的三角形结构相比，本场景变换更多、概率分配不均匀，但仍能由同一套随机迭代流程得到清晰吸引子，说明概率加权抽样适用于形态更复杂的二维 IFS。

## 4 问题记录

首轮渲染结果中，Sierpinski triangle 与 Barnsley's fern 整体落在图像左上区域，与预期的左下构图不符。原因在于像素映射阶段对纵坐标额外做了  $y \leftarrow (1 - v_y)$  翻转，而图像库在写出 TGA 时却已经按内存中  $(0, 0)$  对应文件左下角的基础进行一次翻转  $y$ ，两次翻转使吸引子上下颠倒。去除翻转后解决。

在 Windows 下初次编译时环境中无可用 g++。在本机上安装了 MSYS2 并配置 MinGW-w64 工具链，并把 mingw64/bin 加入用户 PATH，再按当前目录结构调整 Makefile 中的包含路径与源文件列表后问题解决，可正常编译并批量生成样例输出。

## 5 总结

本实验实现了二维 IFS 的读入与随机迭代绘制：以压缩仿射变换及选择概率描述自相似结构，在单位正方形上采样并按累积概率迭代，将结果映射到图像。通过改变迭代次数，可从噪声逐步观察到吸引子成形；在更高采样点数下，同一流程亦可绘制 Barnsley's fern。实现过程中需保证变换与概率的正确读入、仿射变换的正确施加，以及对象销毁与重复读入时的内存释放。

## 附录 (TODO list 记录)

- ☒ 在 Linux / Windows 下编译运行迭代函数系统程序
- ☒ 在 IFS 模块中存储变换个数、仿射矩阵与选择概率，并从文件读入
- ☒ 在 IFS 模块中实现随机迭代渲染，对单位正方形内的随机起点按累积概率迭代仿射变换并绘制到图像
- ☒ 在 IFS 模块析构时释放矩阵与概率占用的内存
- ☒ 在主程序中解析命令行参数，完成读入、渲染与 TGA 图像保存
- ☒ 使用线性代数库表示采样点与仿射变换
- ☐ Extra credit 中自建新的 IFS、调整概率、确定包围盒、改变配色、抗锯齿及深度优先与广度优先等扩展，并在说明文档中简述